

B.TECH (CSE) – 8th SEMESTER

COMPILER DESIGN – ASSIGNMENT

Topic: Implementation of Lexical Analyzer

Submitted by: AmarahMir (210301CSE)

Objective: To design and implement a Lexical Analyzer that recognizes keywords, identifiers, numbers, and operators from a given input string.

Theory: A lexical analyzer (also known as a lexer or scanner) is the first phase of a compiler. It takes the source code as input and converts it into a stream of tokens. Each token represents a logical unit such as a keyword, identifier, constant, or operator. The lexical analyzer removes whitespace and comments from the source code and provides meaningful tokens to the syntax analyzer (parser).

Algorithm:

1. Read the input source code as a string. 2. Scan each character from left to right. 3. Classify characters into categories: letter, digit, operator, or special symbol. 4. Combine sequences of letters and digits into identifiers or keywords. 5. Recognize numbers and operators as separate tokens. 6. Display the tokens and their types.

Program (Python):

```
# Lexical Analyzer in Python
import re
def lexical_analyzer(code):
    tokens = []
    keywords = {'if', 'else', 'while', 'return', 'int', 'float', 'char'}
    token_spec = [
        ('NUMBER', r'\d+(\.\d*)?'),
        ('ID', r'[A-Za-z_]\w*'),
        ('OP', r'[+ \- * / = < > !]'),
        ('NEWLINE', r'\n'),
        ('SKIP', r'[ \t]+'),
        ('MISMATCH', r'.'),
    ]
    tok_regex = '|'.join('(?P<%s>%s)' % pair for pair in token_spec)
    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group()
        if kind == 'ID' and value in keywords:
            kind = 'KEYWORD'
        elif kind == 'SKIP' or kind == 'NEWLINE':
            continue
        elif kind == 'MISMATCH':
            raise RuntimeError(f'Unexpected character: {value!r}')
        tokens.append((kind, value))
    return tokens

# Example Input code = "int a = 5; if (a > 2) a = a + 1;"
for t in lexical_analyzer(code):
    print(t)
```

Sample Output:

```
('KEYWORD', 'int') ('ID', 'a') ('OP', '=') ('NUMBER', '5') ('KEYWORD', 'if') ('ID', 'a') ('OP', '>')
('NUMBER', '2') ('ID', 'a') ('OP', '=') ('ID', 'a') ('OP', '+') ('NUMBER', '1')
```

Result: The lexical analyzer was successfully implemented to identify and classify tokens from the given source code.